CS 25000 Lab 08 THU – Upload to Gradescope by the end of your lab session.

Lab08 Q1. Why do most processor chips manufactured today have a pipelined processor circuit? How does this technology change affect application software developers?

Pipelining allows in all cases multiple instructions to overlap in execution, this makes the increasing instruction throughput and overall performance without increasing clock speed. Each instruction stage is processed and managed concurrently for different instructions. Developers must be aware of instruction dependencies, branch penalties and memory latency that can cause pipeline stalls. So, writing code with fewer dependencies and better instruction level of parallelism improves efficiency.

Lab08 Q2. When a procedure (callee) has more arguments than there are registers available for passing arguments, then the caller passes the excess arguments by placing them on the stack. Which great idea of computer architecture argues to adopt the programming convention of using enough registers for argument passing so that nearly all callees have no excess arguments?

Making the common case fast, most procedures have only few arguments, so providing enough registers for argument passing optimizes the common case, minimizing slower memory accesses via the stack.

Lab08 Q3. The table here repeats some rows of zyBook Figure 2.8.4 LEGv8 Register Conventions with an additional column "Reason not preserved." Give the reason that the registers of each of these rows are not preserved by the callee.

Name	Register number	Usage	Preserved on call?	Reason not preserved
X0 – X7	0-7	Arguments/Results	No	Values are overwritten with new arguments or return values for each call
X8	8	Indirect result location register	No	It is used to pass return address for large structures, so its value changes per call
X9 – X15	9-15	Temporaries	No	Temporaries are caller saved, this can freely use them without saving or restoring.

Lab08 Q4. Given zyBook Figure 2.8.4 LEGv8 Register Conventions and assume a callee that is written in LEGv8 assembly and uses register X20. Will this callee contain assembly language instruction(s) to store to the stack? If so, why? If so, what instruction sequence will be used to push and what instruction sequence to pop?

Yes, register x20 is callee saved register, the callee must save its old value on the stack before using it and restore it before returning.

Push: STUR X20 [SP, #-16]

Pop: LDUR X20, [SP], #0

Lab08 Q5. Write two LEGv8 assembly language integer subtraction instructions (mnemonic SUB) that satisfy the following constraints. Both instructions use register X9, together the instructions contain one true data dependence carried by register X7, contain one anti-dependence carried by register X8, and use no other registers.

SUB X7, X8, X9 - x7 depends on x8 and x9 (RAW)

SUB X9, X7, X8 - true dependence on x7, anti-dependence on x8 (WAR)

Lab08 Q6. The following four-tuples, (source instruction mnemonic, destination instruction mnemonic, dependence register, dependence type) list all the data dependences in a LEGv8 assembly language code snippet. In that snippet, each mnemonic appears just once. All instructions Register-to-Register type and have the form:

mnemonic Result_Reg, Source1_Reg, Source2_Reg.

Write as much of the code snippet as is defined by the following tuple set. For a register that could be either Source1_Reg or Source2_Reg assign that register to the leftmost undefined position. [Hint: some instructions may not be completely specified by the tuple set. For those instructions leave out unknown register(s).]

Tuples:

(AND, XOR, X6, WAW)
(ADD, SUB, X4, WAR)
(ADD, AND, X1, RAW)
(AND, XOR, X6, WAR)
(ADD, XOR, X1, RAW)
(SUB, AND, X6, WAR)

ADD X4, X1, X2
SUB X6, X4, X3
AND X6, X1, X3
XOR X1, X6, X7

Lab08 Q7. Consider the code snippet

```
ADD X1, X2, X3
AND X4, X1, X6
SUB X5, X1, X8
```

Assume that the time needed by the chosen processor hardware circuit to transfer the data of a true data dependence is one clock cycle. Assume that all instructions pass through the ALU stage in one clock cycle. Write the code snippet with all No-Operation (NOP) instructions necessary to account for the 1 clock cycle time delay necessary for the assumed processor circuit.

```
ADD X1, X2, X3
NOP
AND X4, X1, X6
NOP
SUB X5, X1, X8
```

Each instruction depending on x1 waits one cycle for the true data dependence to resolve.

Lab08 Q8. Two assembly language instructions that have no data dependence (none of RAW, WAR, or WAW) may be executed in either order or simultaneously without violating a dependence. For the code snippet of Question Q7, is it possible to schedule the instructions (reorder them without violating existing dependences) so that no time-wasting NOP instruction(s) are necessary.

No, AND and SUB depend on ADD result in x1, so neither can execute earlier, Reordering cannot eliminate the dependences or remove NOP.

Lab08 Q9. Using two LEGv8 data transfer type instructions, write an assembly language code snippet for which it cannot be determined before run time if there is or is not a true data dependence. Do not include any other type of data dependence in your answer.

```
LDUR X1, [X2]
STUR X3, [X2]
```

If x2 points to the same address during execution, a true RAW dependence exists, otherwise there is none it cannot be determined before runtime.

Lab08 Q10. Which type(s) of data dependence can be eliminated by reallocating registers or having more registers?

Anti-dependences WAR and output dependences WAW can be eliminated by assigning different registers, since they arise only from register name reuse not from actual data flow.